

Meeting Note 08

Date: 2026-02-05 **Time:** 14:00 - 14:30 **Location:** Supervision Office / Online **Attendees:** LIN Zhanzhi, CAO Yixin (Supervisor)

1. Objectives

- Review the results of the "Memory-Aware Scheduling" (Sticky Victim) implementation.
- Analyze why the speedup has plateaued at ~4.4x for 16 threads.
- Decide on the strategy for the next optimization phase (Hybrid approach).

2. Progress Report

- **Memory-Aware Scheduling Implemented:**
 - Updated the work-stealing logic in `include/dpqs/parallel/threadpool.hpp`.
 - Replaced the random/round-robin victim selection with a "Sticky Victim" strategy.
 - Logic: A thief "remembers" the last victim it stole from and checks them first on the next attempt, aiming to exploit spatial task locality in the recursion tree.
- **Benchmark Results (10M Integers):**
 - **2 Threads:** 1.86x Speedup (Slight improvement).
 - **16 Threads:** 4.41x Speedup (0% gain vs previous version).
 - **Conclusion:** The heuristic improved low-thread efficiency slightly but failed to break the "Memory Wall" at high thread counts.

3. Discussion & Feedback

Key Discussion Points

- **The 16-Thread Plateau:** Detailed discussion on why scheduling heuristics are insufficient. The Supervisor agreed that at 16 threads, the memory bus is completely saturated. Changing *who* steals the task doesn't change the fact that the data must be moved across the bus.
- **Verification of "Memory Wall":** The flat scaling curve from 8 to 16 threads is the definitive signature of memory bandwidth saturation (or excessive coherence traffic).

Supervisor Feedback

The supervisor noted that a 0% gain result is still a valid and important scientific result. It proves that the bottleneck is no longer in the *search* for work, but in the execution (memory access) or the synchronization overhead itself. Recommended pivoting to **Hybrid Parallelism** (switching to purely sequential code at the leaves) to eliminate the synchronization overhead entirely for the bottom of the tree.

4. Issues & Challenges

- **Hardware Limitation:** The current testbed's memory bandwidth is likely the hard limit.
- **Plan Adjustment:** We need to be realistic about the maximum possible speedup on this machine. We should focus on efficiency (cpu time) rather than just raw wall-clock time if scaling stops.

5. Action Items

- ☒ Analyze Memory-Aware Scheduling results (Completed).
- ☐ Implement **Hybrid Parallelism (Depth-Based Cutoff)**. (Due: Next Meeting)
 - Add logic to switch to `sort_sequential` if recursion depth > K.
 - Tune `MIN_PARALLEL_SORT_SIZE` down (e.g., 8k) to increase load balancing potential before the cutoff.
- ☐ Rerun full benchmarks on the Hybrid implementation.

6. Next Meeting Plan

- **Target Date:** 2026-02-12
- **Focus:** Review Hybrid Parallelism Results & Final Report Preparation.